

CS102: COMPUTER PROGRAMMING II

“Car Rental System”

Project Report

Team Members:

- 1- KHALED KHATIB - 224110393
- 2- ABDULRAHMAN ELMUTAWI - 224110547
- 3- SUHAIL ABUOBAID - 224110256

Dr. Elmoatez Billah Nagoudi

Submission Date

02-05-2026



1. Introduction

The Vehicle Rental System is a Java-based application developed to manage the process of renting vehicles efficiently. The system supports different user roles, including customers, staff, and administrators, each with specific functionalities.

The main objective of the system is to allow customers to view available vehicles, make bookings, cancel reservations, and view their rental history. Staff members can manage bookings, update vehicle availability, and adjust pricing. Administrators have full control over managing users, vehicles, and business analytics.

This project demonstrates key Object-Oriented Programming (OOP) concepts such as encapsulation, inheritance, polymorphism, and abstraction by organizing the system into well-structured classes like Vehicle, Customer, Booking, and RentalSystem.



2. Tools & Technologies

Java (Core Java)

- Used to implement the entire system logic.
- Applied OOP principles like inheritance (Vehicle subclasses), encapsulation (private attributes with getters/setters), and polymorphism.

Java Collections (ArrayList)

- Used to store and manage lists of vehicles, customers, staff, and bookings.

File Handling (BufferedReader / BufferedWriter)

- Used to store and retrieve data from:
 - vehicles.txt
 - users.txt
 - bookings.txt
- Ensures data persistence across sessions.

IDE (NetBeans)

- Used for writing, debugging, and running the code.

Text Files

- Used as a simple database for storing system data such as vehicles, users, and bookings.

AI Tools (ChatGPT)

- Used to assist in debugging, improving logic, and explaining concepts.



3. System Overview

The system is divided into several main components:

1. Vehicle Class (Abstract)

- Contains shared attributes such as vehicle ID, type, capacity, fuel type, and availability.
- Subclasses include Economy, SUV, Luxury, and Electric, demonstrating inheritance.

2. Customer Class

- Stores customer personal and license information.
- Includes validation (e.g., phone format, email format).

3. Booking Class

- Manages booking details such as dates, status, and links customer and vehicle.

4. RentalSystem Class

- Core of the system.
- Handles:
 - Booking creation and cancellation
 - Conflict checking
 - Vehicle reassignment
 - Revenue calculation
 - Analytics (occupancy, popular vehicle type)

5. DataManager Class

- Responsible for loading and saving data from text files.
- Ensures system-level requirement of data consistency across sessions.

4. Implementation Steps

1. Set up the development environment and project structure.
2. Designed the class structure based on OOP principles (Vehicle, Customer, Booking, etc.).
3. Implemented the abstract Vehicle class and its subclasses.
4. Developed Customer, Staff, and Admin classes.
5. Built the Booking system with validation and conflict checking.
6. Implemented the RentalSystem class to manage all operations.
7. Added file handling using DataManager to save/load data.
8. Created test data in text files (vehicles, users, bookings).
9. Tested all functionalities such as booking, cancellation, and updates.
10. Fixed errors and improved validation and logic.



5. Team Member Contributions

ABDULRAHMAN ELMUTAWI – 224110547:

- Implemented Vehicle classes and subclasses
- Worked on availability and booking logic

KHALID KHATIB – 224110393:

- Implemented Customer and Booking classes
- Added validation and rental history features

SUHAIL ABUOBAID – 224110256:

- Developed DataManager
- Implemented file handling and analytics features

6. Screenshots

Main Menu:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====

===== MAIN MENU =====
1. Customer
2. Staff
3. Admin
4. Exit
Enter your choice:
```

Customer POV:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====

===== MAIN MENU =====
1. Customer
2. Staff
3. Admin
4. Exit
Enter your choice: 1

Enter your Customer ID: c1

===== CUSTOMER MENU =====
1. View Available Vehicles
2. Book a Vehicle
3. View Rental History
4. Cancel Reservation
5. View Customer Information
6. Update Profile
7. Back to Main Menu
Enter your choice:
```

Viewing Available Vehicles:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====

===== MAIN MENU =====
1. Customer
2. Staff
3. Admin
4. Exit
Enter your choice: 1

Enter your Customer ID: c1

===== CUSTOMER MENU =====
1. View Available Vehicles
2. Book a Vehicle
3. View Rental History
4. Cancel Reservation
5. View Customer Information
6. Update Profile
7. Back to Main Menu
Enter your choice: 1

===== AVAILABLE VEHICLES =====
ID: EC1 | Plate: RMA1515 | Type: Economy | Capacity: 5 | Transmission: Automatic | Fuel: Gas | Daily Rate: 100.00 | Sta
ID: EC2 | Plate: ABC111 | Type: Economy | Capacity: 5 | Transmission: Manual | Fuel: Gas | Daily Rate: 90.00 | Status: .
ID: EC3 | Plate: KSA303 | Type: Economy | Capacity: 4 | Transmission: Automatic | Fuel: Gas | Daily Rate: 95.00 | Statu
ID: SUV1 | Plate: RYD2020 | Type: SUV | Capacity: 7 | Transmission: Automatic | Fuel: Gas | Daily Rate: 180.00 | Status
ID: SUV3 | Plate: DMM7070 | Type: SUV | Capacity: 6 | Transmission: Automatic | Fuel: Gas | Daily Rate: 175.00 | Status
ID: LX1 | Plate: VIP999 | Type: Luxury | Capacity: 5 | Transmission: Automatic | Fuel: Gas | Daily Rate: 300.00 | Statu
ID: LX3 | Plate: LUX555 | Type: Luxury | Capacity: 4 | Transmission: Automatic | Fuel: Gas | Daily Rate: 350.00 | Statu
ID: EL1 | Plate: ELE123 | Type: Electric | Capacity: 5 | Transmission: Automatic | Fuel: Electric | Daily Rate: 150.00
ID: EL2 | Plate: ELE456 | Type: Electric | Capacity: 5 | Transmission: Automatic | Fuel: Electric | Daily Rate: 160.00
ID: EL3 | Plate: EV900 | Type: Electric | Capacity: 4 | Transmission: Automatic | Fuel: Electric | Daily Rate: 140.00 |

===== CUSTOMER MENU =====
1. View Available Vehicles
2. Book a Vehicle
3. View Rental History
4. Cancel Reservation
5. View Customer Information
6. Update Profile
7. Back to Main Menu
Enter your choice:
```


Booking a Vehicle:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====

===== MAIN MENU =====
1. Customer
2. Staff
3. Admin
4. Exit
Enter your choice: 1

Enter your Customer ID: c1

===== CUSTOMER MENU =====
1. View Available Vehicles
2. Book a Vehicle
3. View Rental History
4. Cancel Reservation
5. View Customer Information
6. Update Profile
7. Back to Main Menu
Enter your choice: 2

===== BOOK A VEHICLE =====
Enter Vehicle ID: EC1
Enter start date (YYYY-MM-DD): 2026-12-21
Enter end date (YYYY-MM-DD): 2026-12-23

Booking created successfully!
Booking ID: B11 | Customer: Khaled | Vehicle: EC1 (Economy) | Start Date: 2026-12-21 | End Date: 2026-12-23 | Status: Upcoming

===== CUSTOMER MENU =====
1. View Available Vehicles
2. Book a Vehicle
3. View Rental History
4. Cancel Reservation
5. View Customer Information
6. Update Profile
7. Back to Main Menu
Enter your choice: |
```



Canceling Bookings:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====

===== MAIN MENU =====
1. Customer
2. Staff
3. Admin
4. Exit
Enter your choice: 1

Enter your Customer ID: c1

===== CUSTOMER MENU =====
1. View Available Vehicles
2. Book a Vehicle
3. View Rental History
4. Cancel Reservation
5. View Customer Information
6. Update Profile
7. Back to Main Menu
Enter your choice: 4

===== CANCEL RESERVATION =====
Enter Booking ID to cancel: B1
Enter cancellation date (YYYY-MM-DD): 2026-12-19
Reservation cancelled successfully.

===== CUSTOMER MENU =====
1. View Available Vehicles
2. Book a Vehicle
3. View Rental History
4. Cancel Reservation
5. View Customer Information
6. Update Profile
7. Back to Main Menu
Enter your choice: |
```



Staff POV:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====

===== MAIN MENU =====
1. Customer
2. Staff
3. Admin
4. Exit
Enter your choice: 2

Enter Staff ID: S1

===== STAFF MENU =====
1. View Customer Bookings
2. Modify Vehicle Availability
3. Modify Vehicle Pricing
4. Apply Seasonal Promotion
5. Reassign Vehicle
6. View Staff Information
7. Back to Main Menu
Enter your choice: |
```

Applying Seasonal Promotions:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====
```

```
===== MAIN MENU =====
```

1. Customer
2. Staff
3. Admin
4. Exit

Enter your choice: 2

Enter Staff ID: S1

```
===== STAFF MENU =====
```

1. View Customer Bookings
2. Modify Vehicle Availability
3. Modify Vehicle Pricing
4. Apply Seasonal Promotion
5. Reassign Vehicle
6. View Staff Information
7. Back to Main Menu

Enter your choice: 4

Enter discount percentage (e.g., 10): 99

Promotion applied to all vehicles.

```
===== STAFF MENU =====
```

1. View Customer Bookings
2. Modify Vehicle Availability
3. Modify Vehicle Pricing
4. Apply Seasonal Promotion
5. Reassign Vehicle
6. View Staff Information
7. Back to Main Menu

Enter your choice: |



Admin POV:

```
=====
WELCOME TO CAR RENTAL SYSTEM
=====

===== MAIN MENU =====
1. Customer
2. Staff
3. Admin
4. Exit
Enter your choice: 3

Enter Admin ID: A1

===== ADMIN MENU =====
1. Manage User Accounts
2. Fleet Management
3. View Business Analytics
4. View Admin Information
5. Back to Main Menu
Enter your choice:
```



7. Source Code

```
import java.util.ArrayList;

import java.io.BufferedReader;

import java.io.BufferedWriter;

import java.io.FileReader;

import java.io.FileWriter;

import java.io.IOException;

import java.util.Scanner;

// ===== VEHICLE CLASS =====

abstract class Vehicle {

    // Vehicle attributes

    private String vehicleID;

    private String plateNumber;

    private int capacity;

    private String transmissionType; // Manual or Automatic

    private String fuelType; // Gas, Electric, or Hybrid

    private double dailyRate;

    private String status; // Available, Rented, or Under Maintenance

    private ArrayList<String> availabilityCalendar; // Available dates

    // Constructor

    public Vehicle(String vehicleID, String plateNumber, int capacity, String transmissionType,
```



```
String fuelType, double dailyRate, String status, ArrayList<String> availabilityCalendar) {

    setVehicleID(vehicleID);
    setPlateNumber(plateNumber);
    setCapacity(capacity);
    setTransmissionType(transmissionType);
    setFuelType(fuelType);
    setDailyRate(dailyRate);
    setStatus(status);
    setAvailabilityCalendar(availabilityCalendar);
}

// Getters and Setters
public String getVehicleID() {
    return vehicleID;
}

public void setVehicleID(String vehicleID) {
    this.vehicleID = vehicleID;
}

public String getPlateNumber() {
    return plateNumber;
}

public void setPlateNumber(String plateNumber) {
```



```
this.plateNumber = plateNumber;
}

public int getCapacity() {
    return capacity;
}

public void setCapacity(int capacity) {
    if (capacity > 0) {
        this.capacity = capacity;
    }
}

public String getTransmissionType() {
    return transmissionType;
}

public void setTransmissionType(String transmissionType) {
    if (transmissionType != null && (transmissionType.equalsIgnoreCase("Manual") ||
transmissionType.equalsIgnoreCase("Automatic"))) {
        this.transmissionType = transmissionType;
    }
}

public String getFuelType() {
    return fuelType;
}
```




```
public void setFuelType(String fuelType) {  
    if (fuelType != null && (fuelType.equalsIgnoreCase("Gas") ||  
fuelType.equalsIgnoreCase("Electric") || fuelType.equalsIgnoreCase("Hybrid"))) {  
        this.fuelType = fuelType;  
    }  
}  
  
public double getDailyRate() {  
    return dailyRate;  
}  
  
public void setDailyRate(double dailyRate) {  
    if (dailyRate > 0) {  
        this.dailyRate = dailyRate;  
    }  
}  
  
public String getStatus() {  
    return status;  
}  
  
public void setStatus(String status) {  
    if (status != null && (status.equalsIgnoreCase("Available") ||  
status.equalsIgnoreCase("Rented") || status.equalsIgnoreCase("Under Maintenance"))) {  
        this.status = status;  
    }  
}
```



```
}
```

```
public ArrayList<String> getAvailabilityCalendar() {  
    return new ArrayList<>(availabilityCalendar);  
}
```

```
public void setAvailabilityCalendar(ArrayList<String> availabilityCalendar) {  
    if (availabilityCalendar != null) {  
        this.availabilityCalendar = new ArrayList<>(availabilityCalendar);  
    }  
}
```

```
// Abstract method: each subclass defines its vehicle type  
public abstract String getVehicleType();
```

```
// Checks availability for the full date range  
public boolean isAvailableForWholeRange(String pickupDate, String returnDate) {  
    if (pickupDate == null || returnDate == null || availabilityCalendar == null) {  
        return false;  
    }  
}
```

```
int pickupIndex = availabilityCalendar.indexOf(pickupDate);  
int returnIndex = availabilityCalendar.indexOf(returnDate);
```

```
if (pickupIndex == -1 || returnIndex == -1) {  
    return false;  
}
```



```
}

if (returnIndex < pickupIndex) {
    return false;
}

return (returnIndex - pickupIndex + 1) <= availabilityCalendar.size();
}

// Remove booked dates from the availability calendar
public ArrayList<String> removeBookedDates(String pickupDate, String returnDate) {
    ArrayList<String> removedDates = new ArrayList<>();

    int pickupIndex = availabilityCalendar.indexOf(pickupDate);
    int returnIndex = availabilityCalendar.indexOf(returnDate);

    if (pickupIndex == -1 || returnIndex == -1 || returnIndex < pickupIndex) {
        return removedDates;
    }

    for (int i = returnIndex; i >= pickupIndex; i--) {
        removedDates.add(0, availabilityCalendar.remove(i));
    }

    return removedDates;
}
```



```
// Add back cancelled dates to the availability calendar
public void addBackDates(ArrayList<String> dates) {
    if (dates != null) {
        for (String d : dates) {
            if (!availabilityCalendar.contains(d)) {
                availabilityCalendar.add(d);
            }
        }
    }

    for (int i = 0; i < availabilityCalendar.size() - 1; i++) {
        for (int j = i + 1; j < availabilityCalendar.size(); j++) {
            if (availabilityCalendar.get(i).compareTo(availabilityCalendar.get(j)) > 0) {
                String temp = availabilityCalendar.get(i);
                availabilityCalendar.set(i, availabilityCalendar.get(j));
                availabilityCalendar.set(j, temp);
            }
        }
    }
}

// Returns available dates as a formatted string
public String getAvailableDates() {
    if (availabilityCalendar == null || availabilityCalendar.isEmpty()) {
        return "No available dates";
    }
}
```



```
}  
    return availabilityCalendar.toString();  
}  
  
@Override  
public String toString() {  
    return "ID: " + vehicleID  
        + " | Plate: " + plateNumber  
        + " | Type: " + getVehicleType()  
        + " | Capacity: " + capacity  
        + " | Transmission: " + transmissionType  
        + " | Fuel: " + fuelType  
        + " | Daily Rate: " + String.format("%.2f", dailyRate)  
        + " | Status: " + status  
        + " | Availability: " + availabilityCalendar;  
}  
}  
  
// ===== VEHICLE TYPES =====  
  
class Economy extends Vehicle {  
  
    // Constructor  
    public Economy(String vehicleID, String plateNumber, int capacity,  
        String transmissionType, String fuelType, double dailyRate,  
        String status, ArrayList<String> availabilityCalendar) {
```



```
        super(vehicleID, plateNumber, capacity, transmissionType, fuelType, dailyRate, status,  
availabilityCalendar);
```

```
    }
```

```
@Override
```

```
public String getVehicleType() {
```

```
    return getClass().getSimpleName();
```

```
}
```

```
}
```

```
class SUV extends Vehicle {
```

```
    // Constructor
```

```
    public SUV(String vehicleID, String plateNumber, int capacity,
```

```
        String transmissionType, String fuelType, double dailyRate,
```

```
        String status, ArrayList<String> availabilityCalendar) {
```

```
        super(vehicleID, plateNumber, capacity, transmissionType, fuelType, dailyRate, status,  
availabilityCalendar);
```

```
    }
```

```
@Override
```

```
public String getVehicleType() {
```

```
    return getClass().getSimpleName();
```

```
}
```



```
}
```

```
class Luxury extends Vehicle {
```

```
    // Constructor
```

```
    public Luxury(String vehicleID, String plateNumber, int capacity,  
        String transmissionType, String fuelType, double dailyRate,  
        String status, ArrayList<String> availabilityCalendar) {
```

```
        super(vehicleID, plateNumber, capacity, transmissionType, fuelType, dailyRate, status,  
availabilityCalendar);
```

```
    }
```

```
    @Override
```

```
    public String getVehicleType() {  
        return getClass().getSimpleName();  
    }
```

```
}
```

```
class Electric extends Vehicle {
```

```
    // Constructor
```

```
    public Electric(String vehicleID, String plateNumber, int capacity,  
        String transmissionType, String fuelType, double dailyRate,  
        String status, ArrayList<String> availabilityCalendar) {
```



```
        super(vehicleID, plateNumber, capacity, transmissionType, fuelType, dailyRate, status,  
availabilityCalendar);
```

```
    }
```

```
@Override
```

```
public String getVehicleType() {
```

```
    return getClass().getSimpleName();
```

```
}
```

```
}
```

```
// ===== CUSTOMER CLASS =====
```

```
class Customer {
```

```
    // Customer attributes
```

```
    private String customerName;
```

```
    private String customerId;
```

```
    private String customerPhoneNumber;
```

```
    private String customerEmail;
```

```
    private String customerDateOfBirth;
```

```
    private String customerAddress;
```

```
    // License information
```

```
    private String licenseNumber;
```

```
    private String licenseType;
```

```
    private String licenseExpiryDate;
```




// Constructor

```
public Customer(String customerName, String customerId, String customerPhoneNumber,  
    String customerEmail, String customerDateOfBirth, String customerAddress,  
    String licenseNumber, String licenseType, String licenseExpiryDate) {  
  
    setCustomerName(customerName);  
    setCustomerId(customerId);  
    setCustomerDateOfBirth(customerDateOfBirth);  
    setCustomerPhoneNumber(customerPhoneNumber);  
    setCustomerEmail(customerEmail);  
    setCustomerAddress(customerAddress);  
    setLicenseNumber(licenseNumber);  
    setLicenseType(licenseType);  
    setLicenseExpiryDate(licenseExpiryDate);  
}
```

// Getters and Setters

```
public String getCustomerName() {  
    return customerName;  
}  
  
public void setCustomerName(String customerName) {  
    if (customerName != null && !customerName.trim().isEmpty()) {  
        this.customerName = customerName;  
    }  
}
```



```
public String getCustomerId() {  
    return customerId;  
}
```

```
public void setCustomerId(String customerId) {  
    if (customerId != null && !customerId.trim().isEmpty()) {  
        this.customerId = customerId;  
    }  
}
```

```
public String getCustomerPhoneNumber() {  
    return customerPhoneNumber;  
}
```

```
// Phone must start with 05 (Saudi format)  
public void setCustomerPhoneNumber(String customerPhoneNumber) {  
    if (customerPhoneNumber != null && !customerPhoneNumber.trim().isEmpty() &&  
customerPhoneNumber.length() >= 9 && customerPhoneNumber.startsWith("05")) {  
        this.customerPhoneNumber = customerPhoneNumber;  
    }  
}
```

```
public String getCustomerEmail() {  
    return customerEmail;  
}
```



```
public void setCustomerEmail(String customerEmail) {  
    if (customerEmail != null && !customerEmail.trim().isEmpty() &&  
customerEmail.contains("@") && customerEmail.contains(".")) {  
        this.customerEmail = customerEmail;  
    }  
}  
  
public String getCustomerDateOfBirth() {  
    return customerDateOfBirth;  
}  
  
public void setCustomerDateOfBirth(String customerDateOfBirth) {  
    if (customerDateOfBirth != null && !customerDateOfBirth.trim().isEmpty()) {  
        this.customerDateOfBirth = customerDateOfBirth;  
    }  
}  
  
public String getCustomerAddress() {  
    return customerAddress;  
}  
  
public void setCustomerAddress(String customerAddress) {  
    if (customerAddress != null && !customerAddress.trim().isEmpty()) {  
        this.customerAddress = customerAddress;  
    }  
}
```



```
public String getLicenseNumber() {  
    return licenseNumber;  
}
```

```
public void setLicenseNumber(String licenseNumber) {  
    if (licenseNumber != null && !licenseNumber.trim().isEmpty()) {  
        this.licenseNumber = licenseNumber;  
    }  
}
```

```
public String getLicenseType() {  
    return licenseType;  
}
```

```
public void setLicenseType(String licenseType) {  
    if (licenseType != null && !licenseType.trim().isEmpty()) {  
        this.licenseType = licenseType;  
    }  
}
```

```
public String getLicenseExpiryDate() {  
    return licenseExpiryDate;  
}
```

```
public void setLicenseExpiryDate(String licenseExpiryDate) {  
    if (licenseExpiryDate != null && !licenseExpiryDate.trim().isEmpty()) {
```



```
this.licenseExpiryDate = licenseExpiryDate;
    }
}

// Returns customer information for display
@Override
public String toString() {
    return "ID: " + customerId
        + " | Name: " + customerName
        + " | Phone: " + customerPhoneNumber
        + " | Email: " + customerEmail
        + " | Date of Birth: " + customerDateOfBirth
        + " | Address: " + customerAddress
        + " | License Number: " + licenseNumber
        + " | License Type: " + licenseType
        + " | License Expiry: " + licenseExpiryDate;
}
}

// ===== BOOKING CLASS =====
class Booking {

    // Booking details and related data
    private Customer customer;
    private Vehicle vehicle;
    private String bookingId;
```



```
private String startDate;

private String endDate;

private String status; // Past, Upcoming, or Cancelled

private String cancelDate; // Set only when booking is cancelled

private ArrayList<String> bookedDates;


// Constructor

public Booking(Customer customer, Vehicle v, String bookingId, String startDate, String
endDate, String status) {

    bookedDates = new ArrayList<>();


    setCustomer(customer);
    setVehicle(v);
    setBookingId(bookingId);
    setStartDate(startDate);
    setEndDate(endDate);
    setStatus(status);
}


// Getters and Setters

public Customer getCustomer() {

    return customer;
}


public void setCustomer(Customer customer) {

    if (customer != null) {

        this.customer = customer;
```



```
    }  
}  
  
public Vehicle getVehicle() {  
    return vehicle;  
}  
  
public void setVehicle(Vehicle vehicle) {  
    if (vehicle != null) {  
        this.vehicle = vehicle;  
    }  
}  
  
public String getBookingId() {  
    return bookingId;  
}  
  
public void setBookingId(String bookingId) {  
    if (bookingId != null && !bookingId.trim().isEmpty()) {  
        this.bookingId = bookingId;  
    }  
}  
  
// Validates date format (YYYY-MM-DD)  
private boolean isValidDateFormat(String date) {  
    return date.matches("\\d{4}-\\d{2}-\\d{2}");  
}
```



```
}
```

```
public String getStartDate() {  
    return startDate;  
}
```

```
public void setStartDate(String startDate) {  
    if (startDate != null && !startDate.trim().isEmpty() && isValidDateFormat(startDate)) {  
        this.startDate = startDate.trim();  
    }  
}
```

```
public String getEndDate() {  
    return endDate;  
}
```

```
public void setEndDate(String endDate) {  
    if (endDate != null && !endDate.trim().isEmpty() && isValidDateFormat(endDate) &&  
startDate != null && endDate.compareTo(startDate) >= 0) {  
        this.endDate = endDate.trim();  
    }  
}
```

```
public String getStatus() {  
    return status;  
}
```




```
public void setStatus(String status) {  
    if (status != null && (status.equalsIgnoreCase("Past") ||  
status.equalsIgnoreCase("Upcoming") || status.equalsIgnoreCase("Cancelled"))) {  
        this.status = status;  
    }  
}  
  
public String getCancelDate() {  
    return cancelDate;  
}  
  
public void setCancelDate(String cancelDate) {  
    if (cancelDate != null) {  
        this.cancelDate = cancelDate;  
    }  
}  
  
public ArrayList<String> getBookedDates() {  
    return new ArrayList<>(bookedDates);  
}  
  
public void setBookedDates(ArrayList<String> bookedDates) {  
    if (bookedDates != null) {  
        this.bookedDates = new ArrayList<>(bookedDates);  
    }  
}
```



```
// Returns booking details for display

@Override
public String toString() {
    String result = "Booking ID: " + bookingId
        + " | Customer: " + customer.getCustomerName()
        + " | Vehicle: " + vehicle.getVehicleID()
        + " (" + vehicle.getVehicleType() + ")"
        + " | Start Date: " + startDate
        + " | End Date: " + endDate
        + " | Status: " + status;

    if ("Cancelled".equalsIgnoreCase(status)) {
        result += " | Cancel Date: " + getCancelDate();
    }

    return result;
}

}

// ===== STAFF CLASS =====
class Staff {

    // Staff attributes
    private String staffId;
    private String name;
```



```
// Constructor

public Staff(String staffId, String name) {

    setStaffId(staffId);

    setName(name);

}


// Getters and Setters

public String getStaffId() {

    return staffId;

}


public void setStaffId(String staffId) {

    if (staffId != null && !staffId.trim().isEmpty()) {

        this.staffId = staffId;

    }

}


public String getName() {

    return name;

}


public void setName(String name) {

    if (name != null && !name.trim().isEmpty()) {

        this.name = name;

    }

}
```



```
}  
  
}  
  
// ===== RENTAL SYSTEM CLASS =====  
  
class RentalSystem {  
  
    // Stores all system data (vehicles, customers, staff, bookings)  
    private ArrayList<Vehicle> vehicles;  
    private ArrayList<Customer> customers;  
    private ArrayList<Staff> staffList;  
    private ArrayList<Booking> bookingList;  
  
    // Constructor  
    public RentalSystem() {  
        vehicles = new ArrayList<>();  
        customers = new ArrayList<>();  
        staffList = new ArrayList<>();  
        bookingList = new ArrayList<>();  
    }  
  
    // Getters and Setters  
    public ArrayList<Vehicle> getVehicles() {  
        return new ArrayList<>(vehicles);  
    }  
  
    public void setVehicles(ArrayList<Vehicle> vehicles) {
```



```
        if (vehicles != null) {
            this.vehicles = vehicles;
        }
    }

    public ArrayList<Customer> getCustomers() {
        return new ArrayList<>(customers);
    }

    public void setCustomers(ArrayList<Customer> customers) {
        if (customers != null) {
            this.customers = customers;
        }
    }

    public ArrayList<Staff> getStaffList() {
        return new ArrayList<>(staffList);
    }

    public void setStaffList(ArrayList<Staff> staffList) {
        if (staffList != null) {
            this.staffList = staffList;
        }
    }

    public ArrayList<Booking> getBookingList() {
```



```
        return new ArrayList<>(bookingList);
    }

    public void setBookingList(ArrayList<Booking> bookingList) {
        if (bookingList != null) {
            this.bookingList = bookingList;
        }
    }

    // Loads all system data from text files
    public void loadData() {
        DataManager dm = new DataManager();

        ArrayList<Vehicle> loadedVehicles = dm.loadVehicles();
        ArrayList<Customer> loadedCustomers = dm.loadCustomers();
        ArrayList<Staff> loadedStaff = dm.loadStaff();
        ArrayList<Booking> loadedBookings = dm.loadBookings(loadedCustomers, loadedVehicles);

        setVehicles(loadedVehicles);
        setCustomers(loadedCustomers);
        setStaffList(loadedStaff);
        setBookingList(loadedBookings);
    }

    // Saves all system data to text files
    public void saveData() {
```



```
DataManager dm = new DataManager();

dm.saveVehicles(vehicles);
dm.saveUsers(customers, staffList);
dm.saveBookings(bookingList);
}

// Check if the same vehicle already has a booking in the selected date range
private boolean hasBookingConflict(Vehicle vehicle, String startDate, String endDate) {
    for (Booking b : bookingList) {

        if (isEqual(b.getVehicle().getVehicleID(), vehicle.getVehicleID())
            && !"Cancelled".equalsIgnoreCase(b.getStatus())) {

            boolean overlap = startDate.compareTo(b.getEndDate()) <= 0
                && endDate.compareTo(b.getStartDate()) >= 0;

            if (overlap) {
                return true;
            }
        }
    }

    return false;
}
```



```
public String generateBookingId() {  
    int max = 0;  
  
    for (Booking b : bookingList) {  
        String id = b.getBookingId().replace(" ", "").toUpperCase();  
  
        if (id.startsWith("B")) {  
            try {  
                int number = Integer.parseInt(id.substring(1));  
                if (number > max) {  
                    max = number;  
                }  
            } catch (NumberFormatException e) {  
                // Ignore invalid booking IDs  
            }  
        }  
    }  
  
    return "B" + (max + 1);  
}  
  
// Creates a new booking if all conditions are met  
public Booking createBooking(Customer customer, Vehicle vehicle, String bookingId,  
    String vehicleType, String startDate, String endDate, String status) {  
  
    if (customer == null || vehicle == null) {
```




```
System.out.println("Invalid customer or vehicle.");  
return null;  
}
```

```
if (startDate == null || endDate == null) {  
    System.out.println("Invalid dates.");  
    return null;  
}
```

```
if (!"Available".equalsIgnoreCase(vehicle.getStatus())) {  
    System.out.println("Vehicle is currently not available.");  
    return null;  
}
```

```
if (vehicleType == null || !vehicleType.equalsIgnoreCase(vehicle.getVehicleType())) {  
    System.out.println("Vehicle type does not match.");  
    return null;  
}
```

```
if (!vehicle.isAvailableForWholeRange(startDate.trim(), endDate.trim()) &&  
    !"Past".equalsIgnoreCase(status)) {  
    System.out.println("Vehicle is not available for these dates.");  
    return null;  
}
```

```
if (hasBookingConflict(vehicle, startDate.trim(), endDate.trim())) {  
    System.out.println("Vehicle already has a booking during this date range.");  
}
```



```
        return null;
    }

    Booking newBooking = new Booking(customer, vehicle, bookingId, startDate, endDate,
status);

    bookingList.add(newBooking);

    if ("Upcoming".equalsIgnoreCase(status)) {

        ArrayList<String> removedDates = vehicle.removeBookedDates(startDate.trim(),
endDate.trim());

        newBooking.setBookedDates(removedDates);

        if (vehicle.getAvailabilityCalendar().isEmpty()) {
            vehicle.setStatus("Rented");
        } else {
            vehicle.setStatus("Available");
        }
    }

    return newBooking;
}

// Cancels booking before pickup date and restores availability
public boolean cancelBooking(Booking booking, String cancelDate) {

    if (booking == null || cancelDate == null) {

        return false;
    }
}
```



```
}

if (!"Upcoming".equalsIgnoreCase(booking.getStatus())) {
    return false;
}

if (cancelDate.compareTo(booking.getStartDate()) >= 0) {
    System.out.println("Cannot cancel after pickup date.");
    return false;
}

booking.setStatus("Cancelled");
booking.setCancelDate(cancelDate);

Vehicle vehicle = booking.getVehicle();
vehicle.addBackDates(booking.getBookedDates());

vehicle.setStatus("Available");

return true;
}

// Reassigns booking to another available vehicle
public boolean reassignVehicle(Booking booking, Vehicle newVehicle) {

    if (booking == null || newVehicle == null) {
```



```
        return false;
    }

    if (!"Upcoming".equalsIgnoreCase(booking.getStatus())) {
        return false;
    }

    if (!"Available".equalsIgnoreCase(newVehicle.getStatus())) {
        System.out.println("New vehicle is not available.");
        return false;
    }

    if (!newVehicle.isAvailableForWholeRange(booking.getStartDate(), booking.getEndDate()))
    {
        System.out.println("New vehicle is not available for the selected dates.");
        return false;
    }

    Vehicle oldVehicle = booking.getVehicle();

    oldVehicle.addBackDates(booking.getBookedDates());
    oldVehicle.setStatus("Available");

    ArrayList<String> removedDates = newVehicle.removeBookedDates(
        booking.getStartDate(),
        booking.getEndDate()
    );
```



```
        booking.setVehicle(newVehicle);
        booking.setBookedDates(removedDates);

        newVehicle.setStatus("Rented");

        return true;
    }

    // Displays customer's rental history (past, upcoming, cancelled)
    public void viewRentalHistory(Customer customer) {

        if (customer == null) {
            System.out.println("Customer not found.");
            return;
        }

        System.out.println();
        System.out.println("===== RENTAL HISTORY =====");
        System.out.println();

        System.out.println("Customer Name : " + customer.getCustomerName());
        System.out.println("Customer ID  : " + customer.getCustomerId());
        System.out.println();

        // Past Rentals
```



```
System.out.println("----- Past Rentals -----");

boolean foundPast = false;

for (Booking b : bookingList) {
    if (isEqual(b.getCustomer().getCustomerId(), customer.getCustomerId()) &&
        "Past".equalsIgnoreCase(b.getStatus())) {
        System.out.println(b);
        foundPast = true;
    }
}

if (!foundPast) {
    System.out.println("No past rentals found.");
}

System.out.println();

// Upcoming Rentals

System.out.println("----- Upcoming Rentals -----");

boolean foundUpcoming = false;

for (Booking b : bookingList) {
    if (isEqual(b.getCustomer().getCustomerId(), customer.getCustomerId()) &&
        "Upcoming".equalsIgnoreCase(b.getStatus())) {
        System.out.println(b);
        foundUpcoming = true;
    }
}
```



```
}

if (!foundUpcoming) {
    System.out.println("No upcoming rentals found.");
}

System.out.println();

// Cancelled Rentals
System.out.println("----- Cancelled Rentals -----");
boolean foundCancelled = false;

for (Booking b : bookingList) {
    if (isEqual(b.getCustomer().getCustomerId(), customer.getCustomerId()) &&
        "Cancelled".equalsIgnoreCase(b.getStatus())) {
        System.out.println(b);
        foundCancelled = true;
    }
}

if (!foundCancelled) {
    System.out.println("No cancelled rentals found.");
}

System.out.println();
System.out.println("=====");
}
```



```
// Update contact information

public void updateContactInfo(Customer customer, String phone, String email, String
birthDate, String address) {

    if (customer != null) {

        customer.setCustomerPhoneNumber(phone);

        customer.setCustomerEmail(email);

        customer.setCustomerDateOfBirth(birthDate);

        customer.setCustomerAddress(address);

    }

}

// Update driver's license information

public void updateLicenseInfo(Customer customer, String number, String type, String
expiryDate) {

    if (customer != null) {

        customer.setLicenseNumber(number);

        customer.setLicenseType(type);

        customer.setLicenseExpiryDate(expiryDate);

    }

}

// Modify vehicle availability

public void updateVehicleAvailability(Vehicle vehicle, String newStatus, ArrayList<String>
newAvailability) {

    vehicle.setStatus(newStatus);

    vehicle.setAvailabilityCalendar(newAvailability);

}
```




```
        System.out.println("Vehicle " + vehicle.getVehicleID() + " status updated to: " + newStatus);
    }

    // Modify vehicle pricing
    public void updateVehiclePricing(Vehicle vehicle, double newDailyRate) {
        if (newDailyRate > 0) {
            vehicle.setDailyRate(newDailyRate);

            System.out.println("Vehicle " + vehicle.getVehicleID() + " price updated to: " +
newDailyRate);
        } else {
            System.out.println("Invalid price.");
        }
    }

    // Apply seasonal promotion
    public void applySeasonalPromotion(Vehicle vehicle, double discountPercentage) {
        if (discountPercentage > 0 && discountPercentage < 100) {
            double newPrice = vehicle.getDailyRate() * (1 - discountPercentage / 100);
            newPrice = Math.round(newPrice * 100.0) / 100.0;
            vehicle.setDailyRate(newPrice);
        }
    }

    // ---- User Account Management ----

    // Add a new customer to the system
    public void addCustomer(Customer newCustomer) {
        if (newCustomer == null) {
```



```
System.out.println("Cannot add a null customer.");  
  
return;  
}  
  
for (Customer c : customers) {  
  
    if (isEqual(c.getCustomerId(), newCustomer.getCustomerId())) {  
        System.out.println("Customer already exists.");  
        return;  
    }  
  
    if (c.getCustomerPhoneNumber().equals(newCustomer.getCustomerPhoneNumber())) {  
        System.out.println("Phone number already used.");  
        return;  
    }  
  
    if (c.getCustomerEmail().equalsIgnoreCase(newCustomer.getCustomerEmail())) {  
        System.out.println("Email already used.");  
        return;  
    }  
}  
  
for (Staff s : staffList) {  
    if (isEqual(s.getStaffId(), newCustomer.getCustomerId())) {  
        System.out.println("This ID is already used by a staff member.");  
        return;  
    }  
}
```



```
    }  
}  
  
    customers.add(newCustomer);  
  
    System.out.println("Customer " + newCustomer.getCustomerName() + " has been  
added.");  
}  
  
// Remove a customer from the system by ID  
public void removeCustomer(String customerId) {  
    Customer toRemove = null;  
  
    for (Customer c : customers) {  
        if (isEqual(c.getCustomerId(), customerId)) {  
            toRemove = c;  
            break;  
        }  
    }  
  
    if (toRemove != null) {  
        customers.remove(toRemove);  
  
        System.out.println("Customer " + toRemove.getCustomerName() + " has been  
removed.");  
    } else {  
        System.out.println("Customer with ID " + customerId + " not found.");  
    }  
}
```



```
// Add a new staff member to the system

public void addStaff(Staff newStaff) {

    if (newStaff == null) {

        System.out.println("Cannot add a null staff member.");

        return;

    }

    for (Customer c : customers) {

        if (isEqual(c.getCustomerId(), newStaff.getStaffId())) {

            System.out.println("This ID is already used by a customer.");

            return;

        }

    }

    for (Staff s : staffList) {

        if (isEqual(s.getStaffId(), newStaff.getStaffId())) {

            System.out.println("Staff already exists.");

            return;

        }

    }

    staffList.add(newStaff);

    System.out.println("Staff member " + newStaff.getName() + " has been added.");

}
```



```
// Update staff account information

public void updateStaff(String staffId, String newName) {

    for (Staff s : staffList) {

        if (isEqual(s.getStaffId(), staffId)) {

            s.setName(newName);

            System.out.println("Staff information updated.");

            return;

        }

    }

    System.out.println("Staff with ID " + staffId + " not found.");

}


// Remove a staff member from the system by ID

public void removeStaff(String staffId) {

    Staff toRemove = null;

    for (Staff s : staffList) {

        if (isEqual(s.getStaffId(), staffId)) {

            toRemove = s;

            break;

        }

    }

    if (toRemove != null) {

        staffList.remove(toRemove);

    }

}
```



```
        System.out.println("Staff member " + toRemove.getName() + " has been removed.");
    } else {
        System.out.println("Staff with ID " + staffId + " not found.");
    }
}

// Add a new vehicle to the fleet
public void addVehicle(Vehicle newVehicle) {
    if (newVehicle != null) {
        vehicles.add(newVehicle);
        System.out.println("Vehicle " + newVehicle.getVehicleID() + " has been added to the
fleet.");
    } else {
        System.out.println("Cannot add a null vehicle.");
    }
}

// Remove a vehicle from the fleet by vehicle ID
public void removeVehicle(String vehicleId) {
    Vehicle toRemove = null;

    for (Vehicle v : vehicles) {
        if (isEqual(v.getVehicleID(), vehicleId)) {
            toRemove = v;
            break;
        }
    }
}
```



```
if (toRemove != null) {  
    vehicles.remove(toRemove);  
  
    System.out.println("Vehicle " + toRemove.getVehicleID() + " has been removed from the  
fleet.");  
} else {  
    System.out.println("Vehicle with ID " + vehicleId + " not found.");  
}  
}
```

// Update a vehicle's specifications

```
public void updateVehicleSpecs(Vehicle vehicle, int newCapacity, String newTransmission,  
    String newFuelType, double newDailyRate) {  
    if (vehicle == null) {  
        System.out.println("Vehicle not found.");  
        return;  
    }
```

```
    vehicle.setCapacity(newCapacity);  
    vehicle.setTransmissionType(newTransmission);  
    vehicle.setFuelType(newFuelType);  
    vehicle.setDailyRate(newDailyRate);
```

```
    System.out.println("Vehicle " + vehicle.getVehicleID() + " specifications updated.");  
}
```

// Calculate total revenue from all Past bookings



```
public double calculateTotalRevenue() {  
    double totalRevenue = 0;  
  
    for (Booking b : bookingList) {  
        if ("Past".equalsIgnoreCase(b.getStatus())) {  
            // Count the number of days between start and end date  
            int days = countDays(b.getStartDate(), b.getEndDate());  
            totalRevenue += days * b.getVehicle().getDailyRate();  
        }  
    }  
  
    return totalRevenue;  
}  
  
// Helper method: count days between two dates in "YYYY-MM-DD" format  
private int countDays(String startDate, String endDate) {  
    // Split dates into year, month, day parts  
    String[] startParts = startDate.split("-");  
    String[] endParts = endDate.split("-");  
  
    int startYear = Integer.parseInt(startParts[0]);  
    int startMonth = Integer.parseInt(startParts[1]);  
    int startDay = Integer.parseInt(startParts[2]);  
  
    int endYear = Integer.parseInt(endParts[0]);  
    int endMonth = Integer.parseInt(endParts[1]);
```




```
int endDay = Integer.parseInt(endParts[2]);

// Convert each date to a total day count using a simple formula
int startTotal = startYear * 365 + startMonth * 30 + startDay;
int endTotal = endYear * 365 + endMonth * 30 + endDay;

int diff = endTotal - startTotal;
return diff > 0 ? diff : 1; // Minimum 1 day
}

// Calculate fleet occupancy rate (percentage of vehicles currently Rented)
public void viewFleetOccupancy() {
    if (vehicles == null || vehicles.isEmpty()) {
        System.out.println("No vehicles in the fleet.");
        return;
    }

    int rentedCount = 0;

    for (Vehicle v : vehicles) {
        if ("Rented".equalsIgnoreCase(v.getStatus())) {
            rentedCount++;
        }
    }

    double occupancyRate = ((double) rentedCount / vehicles.size()) * 100;
```



```
System.out.println();  
System.out.println("----- FLEET OCCUPANCY -----");  
System.out.println("\nTotal Vehicles : " + vehicles.size());  
System.out.println("Rented      : " + rentedCount);  
System.out.println("Occupancy Rate : " + String.format("%.1f", occupancyRate) + "%");  
  
}
```

// Find and display the most popular vehicle type based on booking count

```
public void viewMostPopularVehicleType() {  
    if (bookingList == null || bookingList.isEmpty()) {  
        System.out.println("No bookings to analyse.");  
        return;  
    }  
}
```

```
int economyCount = 0;
```

```
int suvCount = 0;
```

```
int luxuryCount = 0;
```

```
int electricCount = 0;
```

```
for (Booking b : bookingList) {
```

```
    String type = b.getVehicle().getVehicleType();
```

```
    if (type.equalsIgnoreCase("Economy")) {
```

```
        economyCount++;
```



```
    } else if (type.equalsIgnoreCase("SUV")) {  
        suvCount++;  
    } else if (type.equalsIgnoreCase("Luxury")) {  
        luxuryCount++;  
    } else if (type.equalsIgnoreCase("Electric")) {  
        electricCount++;  
    }  
}
```

```
int highestCount = economyCount;
```

```
if (suvCount > highestCount) {  
    highestCount = suvCount;  
}  
if (luxuryCount > highestCount) {  
    highestCount = luxuryCount;  
}  
if (electricCount > highestCount) {  
    highestCount = electricCount;  
}
```

```
String mostPopular = "";
```

```
if (economyCount == highestCount) {  
    mostPopular += "Economy ";  
}
```



```
        if (suvCount == highestCount) {
            mostPopular += "SUV ";
        }
        if (luxuryCount == highestCount) {
            mostPopular += "Luxury ";
        }
        if (electricCount == highestCount) {
            mostPopular += "Electric ";
        }

        System.out.println();
        System.out.println("----- VEHICLE POPULARITY -----");
        System.out.println("Economy : " + economyCount + " booking(s)");
        System.out.println("SUV    : " + suvCount + " booking(s)");
        System.out.println("Luxury  : " + luxuryCount + " booking(s)");
        System.out.println("Electric : " + electricCount + " booking(s)");
        System.out.println("Most Popular Type: " + mostPopular.trim() + " (" + highestCount + " booking(s))");
    }

    // Display a full analytics report
    public void viewBusinessAnalytics() {
        System.out.println();
        System.out.println("===== BUSINESS ANALYTICS =====");

        double revenue = calculateTotalRevenue();
```



```
System.out.println("\nTotal Revenue from Past Bookings: " + String.format("%.2f",  
revenue) + " SAR");
```

```
viewFleetOccupancy();
```

```
viewMostPopularVehicleType();
```

```
System.out.println("\n=====");
```

```
}
```

```
private boolean isEqual(String a, String b) {
```

```
    return a.replace(" ", "").equalsIgnoreCase(b.replace(" ", ""));
```

```
}
```

```
}
```

```
// ===== ADMIN CLASS =====
```

```
class Admin {
```

```
    // Admin attributes
```

```
    private String adminId;
```

```
    private String name;
```

```
    // Constructor
```

```
    public Admin(String adminId, String name) {
```

```
        setAdminId(adminId);
```

```
        setName(name);
```

```
}
```



```
// Getters and Setters

public String getAdminId() {
    return adminId;
}

public void setAdminId(String adminId) {
    if (adminId != null && !adminId.trim().isEmpty()) {
        this.adminId = adminId;
    }
}

public String getName() {
    return name;
}

public void setName(String name) {
    if (name != null && !name.trim().isEmpty()) {
        this.name = name;
    }
}

}

// ===== DATA MANAGER CLASS =====
// Handles saving and loading data to/from text files
class DataManager {
```



```
// File paths

private static final String VEHICLES_FILE =
"C:\\Users\\obmkk\\OneDrive\\Desktop\\vehicles.txt";

private static final String USERS_FILE = "C:\\Users\\obmkk\\OneDrive\\Desktop\\users.txt";

private static final String BOOKINGS_FILE =
"C:\\Users\\obmkk\\OneDrive\\Desktop\\bookings.txt";


// ---- SAVE METHODS ----

// Save all vehicles to vehicles.txt

// Format:
vehicleID|plateNumber|vehicleType|capacity|transmissionType|fuelType|dailyRate|status|date1,date2,...

public void saveVehicles(ArrayList<Vehicle> vehicles) {
    try {
        BufferedWriter writer = new BufferedWriter(new FileWriter(VEHICLES_FILE));

        for (Vehicle v : vehicles) {
            // Build the comma-separated list of available dates
            ArrayList<String> dates = v.getAvailabilityCalendar();
            StringBuilder dateStr = new StringBuilder();
            for (int i = 0; i < dates.size(); i++) {
                dateStr.append(dates.get(i));
                if (i < dates.size() - 1) {
                    dateStr.append(",");
                }
            }
        }
    }
}
```



```
writer.write(v.getVehicleID() + "|"
    + v.getPlateNumber() + "|"
    + v.getVehicleType() + "|"
    + v.getCapacity() + "|"
    + v.getTransmissionType() + "|"
    + v.getFuelType() + "|"
    + v.getDailyRate() + "|"
    + v.getStatus() + "|"
    + dateStr.toString());
writer.newLine();
}

writer.close();

} catch (IOException e) {
    System.out.println("Error saving vehicles: " + e.getMessage());
}
}

// Save all customers and staff to users.txt
// Customer format:
CUSTOMER|name|id|phone|email|dob|address|licenseNum|licenseType|licenseExpiry
// Staff format: STAFF|id|name
public void saveUsers(ArrayList<Customer> customers, ArrayList<Staff> staffList) {
    try {
        BufferedWriter writer = new BufferedWriter(new FileWriter(USERS_FILE));
```




```
for (Customer c : customers) {  
    writer.write("CUSTOMER|" +  
        + c.getCustomerName() + "|" +  
        + c.getCustomerId() + "|" +  
        + c.getCustomerPhoneNumber() + "|" +  
        + c.getCustomerEmail() + "|" +  
        + c.getCustomerDateOfBirth() + "|" +  
        + c.getCustomerAddress() + "|" +  
        + c.getLicenseNumber() + "|" +  
        + c.getLicenseType() + "|" +  
        + c.getLicenseExpiryDate());  
    writer.newLine();  
}  
  
for (Staff s : staffList) {  
    writer.write("STAFF|" +  
        + s.getStaffId() + "|" +  
        + s.getName());  
    writer.newLine();  
}  
  
writer.close();  
  
} catch (IOException e) {  
    System.out.println("Error saving users: " + e.getMessage());  
}
```



```
}
```

```
// Save all bookings to bookings.txt
```

```
// Format: bookingId|customerId|vehicleId|startDate|endDate|status|cancelDate
```

```
// cancelDate is written as "none" if the booking was not cancelled
```

```
public void saveBookings(ArrayList<Booking> bookings) {
```

```
    try {
```

```
        BufferedWriter writer = new BufferedWriter(new FileWriter(BOOKINGS_FILE));
```

```
        for (Booking b : bookings) {
```

```
            String cancelDate = (b.getCancelDate() != null) ? b.getCancelDate() : "none";
```

```
            writer.write(b.getBookingId() + "|"
```

```
                + b.getCustomer().getCustomerId() + "|"
```

```
                + b.getVehicle().getVehicleID() + "|"
```

```
                + b.getStartDate() + "|"
```

```
                + b.getEndDate() + "|"
```

```
                + b.getStatus() + "|"
```

```
                + cancelDate);
```

```
            writer.newLine();
```

```
        }
```

```
        writer.close();
```

```
    } catch (IOException e) {
```

```
        System.out.println("Error saving bookings: " + e.getMessage());
```



```
}  
}
```

```
// Load vehicles from vehicles.txt and return them as an ArrayList
```

```
public ArrayList<Vehicle> loadVehicles() {
```

```
    ArrayList<Vehicle> vehicles = new ArrayList<>();
```

```
    try {
```

```
        BufferedReader reader = new BufferedReader(new FileReader(VEHICLES_FILE));
```

```
        String line;
```

```
        while ((line = reader.readLine()) != null) {
```

```
            line = line.trim();
```

```
            if (line.isEmpty()) {
```

```
                continue;
```

```
            }
```

```
            String[] parts = line.split("\\|");
```

```
            // We need at least 9 fields (the 9th being the dates, which can be empty)
```

```
            if (parts.length < 8) {
```

```
                continue;
```

```
            }
```

```
            String vehicleID = parts[0];
```

```
            String plateNumber = parts[1];
```



```
String vehicleType = parts[2];

int capacity = Integer.parseInt(parts[3]);

String transmissionType = parts[4];

String fuelType = parts[5];

double dailyRate = Double.parseDouble(parts[6]);

String status = parts[7];


// Parse available dates (comma-separated in parts[8])
ArrayList<String> dates = new ArrayList<>();
if (parts.length >= 9 && !parts[8].trim().isEmpty()) {
    String[] dateArray = parts[8].split(",");
    for (String d : dateArray) {
        dates.add(d.trim());
    }
}


// Create the correct subclass based on the type field
Vehicle vehicle = null;

if (vehicleType.equalsIgnoreCase("Economy")) {
    vehicle = new Economy(vehicleID, plateNumber, capacity, transmissionType,
fuelType, dailyRate, status, dates);
} else if (vehicleType.equalsIgnoreCase("SUV")) {
    vehicle = new SUV(vehicleID, plateNumber, capacity, transmissionType, fuelType,
dailyRate, status, dates);
} else if (vehicleType.equalsIgnoreCase("Luxury")) {
    vehicle = new Luxury(vehicleID, plateNumber, capacity, transmissionType, fuelType,
dailyRate, status, dates);
} else if (vehicleType.equalsIgnoreCase("Electric")) {
```



```
        vehicle = new Electric(vehicleID, plateNumber, capacity, transmissionType, fuelType,
dailyRate, status, dates);
    }

    if (vehicle != null) {
        vehicles.add(vehicle);
    }
}

reader.close();

} catch (IOException e) {
    System.out.println("Could not load vehicles: " + e.getMessage());
}

return vehicles;
}

// Load customers from users.txt and return them as an ArrayList
public ArrayList<Customer> loadCustomers() {
    ArrayList<Customer> customers = new ArrayList<>();

    try {
        BufferedReader reader = new BufferedReader(new FileReader(USERS_FILE));
        String line;

        while ((line = reader.readLine()) != null) {
```



```
line = line.trim();  
if (line.isEmpty()) {  
    continue;  
}  
  
String[] parts = line.split("\\|");  
  
// Only process lines that start with CUSTOMER and have 10 fields  
if (parts.length < 10 || !parts[0].equalsIgnoreCase("CUSTOMER")) {  
    continue;  
}  
  
Customer c = new Customer(  
    parts[1], // name  
    parts[2], // id  
    parts[3], // phone  
    parts[4], // email  
    parts[5], // dob  
    parts[6], // address  
    parts[7], // licenseNumber  
    parts[8], // licenseType  
    parts[9] // licenseExpiry  
);  
  
customers.add(c);  
}
```



```
        reader.close();

    } catch (IOException e) {

        System.out.println("Could not load customers: " + e.getMessage());

    }

    return customers;

}

// Load staff from users.txt and return them as an ArrayList
public ArrayList<Staff> loadStaff() {

    ArrayList<Staff> staffList = new ArrayList<>();

    try {

        BufferedReader reader = new BufferedReader(new FileReader(USERS_FILE));

        String line;

        while ((line = reader.readLine()) != null) {

            line = line.trim();

            if (line.isEmpty()) {

                continue;

            }

            String[] parts = line.split("\\|");
```



```
// Only process lines that start with STAFF and have 3 fields
if (parts.length < 3 || !parts[0].equalsIgnoreCase("STAFF")) {
    continue;
}

Staff s = new Staff(parts[1], parts[2]);
staffList.add(s);
}

reader.close();

} catch (IOException e) {
    System.out.println("Could not load staff: " + e.getMessage());
}

return staffList;
}

// Load bookings from bookings.txt
// Requires already-loaded customers and vehicles lists to link by ID
public ArrayList<Booking> loadBookings(ArrayList<Customer> customers, ArrayList<Vehicle>
vehicles) {
    ArrayList<Booking> bookings = new ArrayList<>();

    try {
        BufferedReader reader = new BufferedReader(new FileReader(BOOKINGS_FILE));
        String line;
```




```
while ((line = reader.readLine()) != null) {  
    line = line.trim();  
    if (line.isEmpty()) {  
        continue;  
    }  
  
    String[] parts = line.split("\\|");  
  
    // We need exactly 7 fields  
    if (parts.length < 7) {  
        continue;  
    }  
  
    String bookingId = parts[0];  
    String customerId = parts[1];  
    String vehicleId = parts[2];  
    String startDate = parts[3];  
    String endDate = parts[4];  
    String status = parts[5];  
    String cancelDate = parts[6];  
  
    // Find the matching Customer object  
    Customer matchedCustomer = null;  
    for (Customer c : customers) {  
        if (c.getCustomerId().equalsIgnoreCase(customerId)) {
```



```
        matchedCustomer = c;
        break;
    }
}

// Find the matching Vehicle object
Vehicle matchedVehicle = null;
for (Vehicle v : vehicles) {
    if (v.getVehicleID().equalsIgnoreCase(vehicleId)) {
        matchedVehicle = v;
        break;
    }
}

// Only create the booking if both customer and vehicle were found
if (matchedCustomer != null && matchedVehicle != null) {
    Booking b = new Booking(matchedCustomer, matchedVehicle, bookingId, startDate,
    endDate, status);

    // Restore cancel date if booking was cancelled
    if (!cancelDate.equalsIgnoreCase("none")) {
        b.setCancelDate(cancelDate);
    }

    bookings.add(b);
}
}
```



```
        reader.close();

    } catch (IOException e) {

        System.out.println("Could not load bookings: " + e.getMessage());

    }

    return bookings;

}

}

// ===== MAIN CLASS =====

public class CarRentalApp {

    // Main application objects
    private RentalSystem system;
    private Scanner input;
    private Admin admin = new Admin("A1", "Elmoataz");

    // Constructor: initializes the system, scanner, and loads data
    public CarRentalApp() {

        system = new RentalSystem();
        input = new Scanner(System.in);
        system.loadData();

    }

}
```



```
// Main method: starts the application

public static void main(String[] args) {

    CarRentalApp app = new CarRentalApp();

    app.start();

}


// Displays the main menu and controls the main program loop
public void start() {

    System.out.println("=====");
    System.out.println(" WELCOME TO CAR RENTAL SYSTEM");
    System.out.println("=====");

    boolean running = true;

    while (running) {

        System.out.println("\n===== MAIN MENU =====");
        System.out.println("1. Customer");
        System.out.println("2. Staff");
        System.out.println("3. Admin");
        System.out.println("4. Exit");
        System.out.print("Enter your choice: ");

        int choice = getIntInput();
```



```
if (choice == -1) {  
    continue;  
}  
  
// Handle main menu options  
switch (choice) {  
    case 1:  
        customerMenu();  
        break;  
  
    case 2:  
        staffMenu();  
        break;  
  
    case 3:  
        adminMenu();  
        break;  
  
    case 4:  
        running = false;  
        break;  
  
    default:  
        System.out.println("Invalid choice. Try again.");  
}  
}
```



```
System.out.println("\nSaving data...");  
system.saveData();  
System.out.println("Goodbye!");  
}  
  
// Handles customer login and customer menu options  
private void customerMenu() {  
  
    System.out.print("\nEnter your Customer ID: ");  
    String customerId = input.nextLine();  
  
    Customer customer = null;  
  
    // Search for customer by ID  
    for (Customer c : system.getCustomers()) {  
        if (isEqual(c.getCustomerId(), customerId)) {  
            customer = c;  
            break;  
        }  
    }  
  
    if (customer == null) {  
        System.out.println("Customer not found.");  
        return;  
    }  
}
```



```
boolean running = true;

while (running) {

    System.out.println("\n===== CUSTOMER MENU =====");
    System.out.println("1. View Available Vehicles");
    System.out.println("2. Book a Vehicle");
    System.out.println("3. View Rental History");
    System.out.println("4. Cancel Reservation");
    System.out.println("5. View Customer Information");
    System.out.println("6. Update Profile");
    System.out.println("7. Back to Main Menu");
    System.out.print("Enter your choice: ");

    int choice = getIntInput();

    if (choice == -1) {
        continue;
    }

    // Handle customer menu options
    switch (choice) {

        case 1:
            viewAvailableVehicles();
```



```
break;

case 2:
    bookVehicle(customer);
    break;

case 3:
    system.viewRentalHistory(customer);
    break;

case 4:
    cancelReservation(customer);
    break;

case 5:
    System.out.println(customer);
    break;

case 6:
    updateProfile(customer);
    break;

case 7:
    running = false;
    break;
```




```
        default:
            System.out.println("Invalid choice.");
        }
    }
}

// Handles staff login and staff menu options
private void staffMenu() {

    System.out.print("\nEnter Staff ID: ");
    String staffId = input.nextLine();

    Staff staff = null;

    // Search for staff by ID
    for (Staff s : system.getStaffList()) {
        if (isEqual(s.getStaffId(), staffId)) {
            staff = s;
            break;
        }
    }

    if (staff == null) {
        System.out.println("Staff not found.");
        return;
    }
}
```



```
boolean running = true;

while (running) {

    System.out.println("\n===== STAFF MENU =====");
    System.out.println("1. View Customer Bookings");
    System.out.println("2. Modify Vehicle Availability");
    System.out.println("3. Modify Vehicle Pricing");
    System.out.println("4. Apply Seasonal Promotion");
    System.out.println("5. Reassign Vehicle");
    System.out.println("6. View Staff Information");
    System.out.println("7. Back to Main Menu");
    System.out.print("Enter your choice: ");

    int choice = getIntInput();

    if (choice == -1) {
        continue;
    }

    // Handle staff menu options
    switch (choice) {
        case 1:
            viewCustomerBookingsByStaff();
            break;
```



case 2:

```
    modifyVehicleAvailability();
```

```
    break;
```

case 3:

```
    modifyVehiclePricing();
```

```
    break;
```

case 4:

```
    applyPromotion();
```

```
    break;
```

case 5:

```
    reassignVehicle();
```

```
    break;
```

case 6:

```
    System.out.println("ID: " + staff.getStaffId() + " | Name: " + staff.getName());
```

```
    break;
```

case 7:

```
    running = false;
```

```
    break;
```

default:



```
        System.out.println("Invalid choice.");
    }
}
}

// Handles admin login and admin menu options
private void adminMenu() {

    System.out.print("\nEnter Admin ID: ");
    String adminId = input.nextLine();

    if (!isEqual(admin.getAdminId(), adminId)) {
        System.out.println("Admin not found.");
        return;
    }

    boolean running = true;

    while (running) {
        System.out.println("\n===== ADMIN MENU =====");
        System.out.println("1. Manage User Accounts");
        System.out.println("2. Fleet Management");
        System.out.println("3. View Business Analytics");
        System.out.println("4. View Admin Information");
        System.out.println("5. Back to Main Menu");
        System.out.print("Enter your choice: ");
```



```
int choice = getIntInput();

if (choice == -1) {
    continue;
}

// Handle admin menu options
switch (choice) {
    case 1:
        manageUserAccounts();
        break;

    case 2:
        fleetManagement();
        break;

    case 3:
        system.viewBusinessAnalytics();
        break;

    case 4:
        System.out.println("ID: " + admin.getAdminId()
            + " | Name: " + admin.getName());
        break;
```



```
        case 5:
            running = false;
            break;

        default:
            System.out.println("Invalid choice.");
    }
}

// Displays all available vehicles that have available dates
private void viewAvailableVehicles() {
    System.out.println("\n===== AVAILABLE VEHICLES =====");

    for (Vehicle v : system.getVehicles()) {
        if ("Available".equalsIgnoreCase(v.getStatus()) && !v.getAvailabilityCalendar().isEmpty())
        {
            System.out.println(v);
        }
    }
}

// Handles vehicle booking process for a customer
private void bookVehicle(Customer customer) {

    System.out.println("\n===== BOOK A VEHICLE =====");
```



```
System.out.print("Enter Vehicle ID: ");

String vehicleId = input.nextLine();

Vehicle selectedVehicle = null;

// Search for selected vehicle
for (Vehicle v : system.getVehicles()) {
    if (isEqual(v.getVehicleID(), vehicleId)) {
        selectedVehicle = v;
        break;
    }
}

if (selectedVehicle == null) {
    System.out.println("Vehicle not found.");
    return;
}

if (!"Available".equalsIgnoreCase(selectedVehicle.getStatus())) {
    System.out.println("Vehicle is currently not available.");
    return;
}

System.out.print("Enter start date (YYYY-MM-DD): ");
String startDate = input.nextLine();
```



```
System.out.print("Enter end date (YYYY-MM-DD): ");

String endDate = input.nextLine();

if (!isValidDateFormat(startDate) || !isValidDateFormat(endDate)) {
    System.out.println("Invalid date format. Use YYYY-MM-DD.");
    return;
}

if (endDate.compareTo(startDate) < 0) {
    System.out.println("End date must be after start date.");
    return;
}

Booking booking = system.createBooking(
    customer,
    selectedVehicle,
    system.generateBookingId(),
    selectedVehicle.getVehicleType(),
    startDate,
    endDate,
    "Upcoming"
);

if (booking != null) {
    System.out.println("\nBooking created successfully!");
    System.out.println(booking);
}
```




```
}  
}  
  
// Handles reservation cancellation for a customer  
private void cancelReservation(Customer customer) {  
  
    System.out.println("\n===== CANCEL RESERVATION =====");  
  
    System.out.print("Enter Booking ID to cancel: ");  
    String bookingId = input.nextLine();  
  
    Booking selectedBooking = null;  
  
    // Find booking for this customer  
    for (Booking b : system.getBookingList()) {  
        if (isEqual(b.getBookingId(), bookingId)  
            && isEqual(b.getCustomer().getCustomerId(), customer.getCustomerId())) {  
            selectedBooking = b;  
            break;  
        }  
    }  
  
    if (selectedBooking == null) {  
        System.out.println("Booking not found for this customer.");  
        return;  
    }  
}
```



```
System.out.print("Enter cancellation date (YYYY-MM-DD): ");  
  
String cancelDate = input.nextLine();  
  
  
boolean result = system.cancelBooking(selectedBooking, cancelDate);  
  
if (result) {  
    System.out.println("Reservation cancelled successfully.");  
}  
}  
  
  
// Allows customer to update contact and license information  
private void updateProfile(Customer customer) {  
  
    System.out.println("\n===== UPDATE PROFILE =====");  
  
    System.out.print("\nEnter new phone: ");  
    String newPhone = input.nextLine();  
  
    System.out.print("Enter new email: ");  
    String newEmail = input.nextLine();  
  
    System.out.print("Enter new address: ");  
    String newAddress = input.nextLine();  
  
    System.out.print("Enter new date of birth: ");
```



```
String newDOB = input.nextLine();

system.updateContactInfo(customer, newPhone, newEmail, newDOB, newAddress);

System.out.print("Enter new license number: ");
String newLicenseNum = input.nextLine();

System.out.print("Enter new license type: ");
String newLicenseType = input.nextLine();

System.out.print("Enter new license expiry: ");
String newExpiry = input.nextLine();

system.updateLicenseInfo(customer, newLicenseNum, newLicenseType, newExpiry);

System.out.println("Profile update completed. Invalid fields were ignored.");

}

// Displays all bookings for staff review
private void viewCustomerBookingsByStaff() {

    System.out.println("\n===== ALL BOOKINGS =====");

    if (system.getBookingList().isEmpty()) {
        System.out.println("No bookings found.");
    }
}
```



```
        return;
    }

    for (Booking b : system.getBookingList()) {
        System.out.println(b);
    }
}

// Allows staff to update vehicle availability status
private void modifyVehicleAvailability() {

    System.out.print("\nEnter Vehicle ID: ");
    String vehicleId = input.nextLine();

    Vehicle vehicle = null;

    // Find vehicle by ID
    for (Vehicle v : system.getVehicles()) {
        if (isEqual(v.getVehicleID(), vehicleId)) {
            vehicle = v;
            break;
        }
    }

    if (vehicle == null) {
        System.out.println("Vehicle not found.");
    }
}
```



```
        return;
    }

    System.out.println("Current status: " + vehicle.getStatus());
    System.out.print("Enter new status (Available / Rented / Under Maintenance): ");
    String newStatus = input.nextLine();

    system.updateVehicleAvailability(vehicle, newStatus, vehicle.getAvailabilityCalendar());
}

// Allows staff to update vehicle daily price
private void modifyVehiclePricing() {

    System.out.print("\nEnter Vehicle ID: ");
    String vehicleId = input.nextLine();

    Vehicle vehicle = null;

    // Find vehicle by ID
    for (Vehicle v : system.getVehicles()) {
        if (isEqual(v.getVehicleID(), vehicleId)) {
            vehicle = v;
            break;
        }
    }
}
```



```
if (vehicle == null) {  
    System.out.println("Vehicle not found.");  
    return;  
}  
  
System.out.println("Current price: " + vehicle.getDailyRate());  
System.out.print("Enter new price: ");  
  
double newPrice;  
  
try {  
    newPrice = Double.parseDouble(input.nextLine());  
} catch (NumberFormatException e) {  
    System.out.println("Invalid input. Please enter a valid price.");  
    return;  
}  
  
if (newPrice <= 0) {  
    System.out.println("Invalid price. Must be greater than 0.");  
    return;  
}  
  
newPrice = Math.round(newPrice * 100.0) / 100.0;  
system.updateVehiclePricing(vehicle, newPrice);  
}  
  
// Applies a discount percentage to all vehicles
```



```
private void applyPromotion() {

    System.out.print("\nEnter discount percentage (e.g., 10): ");

    int discount;

    try {
        discount = Integer.parseInt(input.nextLine());
    } catch (NumberFormatException e) {
        System.out.println("Invalid input. Please enter a whole number.");
        return;
    }

    if (discount <= 0 || discount >= 100) {
        System.out.println("Invalid discount. Must be between 0 and 100.");
        return;
    }

    for (Vehicle v : system.getVehicles()) {
        system.applySeasonalPromotion(v, discount);
    }

    System.out.println("Promotion applied to all vehicles.");
}

// Allows staff to assign a booking to another available vehicle
```



```
private void reassignVehicle() {

    System.out.print("\nEnter Booking ID: ");
    String bookingId = input.nextLine();

    if (!bookingId.toUpperCase().startsWith("B")) {
        bookingId = "B" + bookingId;
    }

    Booking booking = null;

    // Find booking
    for (Booking b : system.getBookingList()) {
        if (isEqual(b.getBookingId(), bookingId)) {
            booking = b;
            break;
        }
    }

    if (booking == null) {
        System.out.println("Booking not found.");
        return;
    }

    System.out.print("Enter new Vehicle ID: ");
    String newVehicleId = input.nextLine();
```




```
Vehicle newVehicle = null;

// Find new vehicle
for (Vehicle v : system.getVehicles()) {
    if (isEqual(v.getVehicleID(), newVehicleId)) {
        newVehicle = v;
        break;
    }
}

if (newVehicle == null) {
    System.out.println("Vehicle not found.");
    return;
}

boolean result = system.reassignVehicle(booking, newVehicle);

if (result) {
    System.out.println("Vehicle reassigned successfully.");
}
}

// Allows admin to add or remove customers and staff
private void manageUserAccounts() {
```



```
boolean running = true;

while (running) {

    System.out.println("\n===== MANAGE USER ACCOUNTS =====");

    System.out.println("1. Add Customer");

    System.out.println("2. Add Staff");

    System.out.println("3. Delete Customer");

    System.out.println("4. Delete Staff");

    System.out.println("5. Update Staff");

    System.out.println("6. Back");

    System.out.print("Enter your choice: ");

    int choice = getIntInput();

    if (choice == -1) {
        continue;
    }

    // Handle user management operations
    switch (choice) {

        case 1:

            System.out.print("Enter name: ");

            String name = input.nextLine();

            System.out.print("Enter ID: ");
```



```
String id = input.nextLine();
```

```
System.out.print("Enter phone: ");
```

```
String phone = input.nextLine();
```

```
System.out.print("Enter email: ");
```

```
String email = input.nextLine();
```

```
System.out.print("Enter Date Of Birth: ");
```

```
String dob = input.nextLine();
```

```
System.out.print("Enter address: ");
```

```
String address = input.nextLine();
```

```
System.out.print("Enter license number: ");
```

```
String licenseNumber = input.nextLine();
```

```
System.out.print("Enter license type: ");
```

```
String licenseType = input.nextLine();
```

```
System.out.print("Enter license expiry: ");
```

```
String expiry = input.nextLine();
```

```
if (!phone.startsWith("05") || phone.length() < 9) {
```

```
    System.out.println("Invalid phone number.");
```

```
    break;
```



```
}

if (!email.contains("@") || !email.contains(".")) {
    System.out.println("Invalid email.");
    break;
}

if (!isValidDateFormat(dob) || !isValidDateFormat(expiry)) {
    System.out.println("Invalid date format. Use YYYY-MM-DD.");
    break;
}

Customer newCustomer = new Customer(
    name, id, phone, email, dob,
    address, licenseNumber, licenseType, expiry
);

system.addCustomer(newCustomer);
break;

case 2:
    System.out.print("Enter staff ID: ");
    String staffId = input.nextLine();

    System.out.print("Enter name: ");
    String staffName = input.nextLine();
```



```
Staff newStaff = new Staff(staffId, staffName);
```

```
system.addStaff(newStaff);
```

```
break;
```

case 3:

```
System.out.print("Enter Customer ID to delete: ");
```

```
String customerId = input.nextLine();
```

```
system.removeCustomer(customerId);
```

```
break;
```

case 4:

```
System.out.print("Enter Staff ID to delete: ");
```

```
String staffIdToRemove = input.nextLine();
```

```
system.removeStaff(staffIdToRemove);
```

```
break;
```

case 5:

```
System.out.print("Enter Staff ID to update: ");
```

```
String updateStaffId = input.nextLine();
```

```
System.out.print("Enter new staff name: ");
```

```
String newStaffName = input.nextLine();
```



```
        system.updateStaff(updateStaffId, newStaffName);

        break;

    case 6:

        running = false;

        break;

    default:

        System.out.println("Invalid choice.");

    }

}

}

// Allows admin to add or remove vehicles from the fleet
private void fleetManagement() {

    boolean running = true;

    while (running) {

        System.out.println("\n===== FLEET MANAGEMENT =====");

        System.out.println("1. Add Vehicle");

        System.out.println("2. Remove Vehicle");

        System.out.println("3. Update Vehicle Specifications");

        System.out.println("4. Back");

        System.out.print("Enter your choice: ");
```



```
int choice = getIntInput();

if (choice == -1) {
    continue;
}

// Handle fleet management operations
switch (choice) {

    case 1:
        System.out.print("Enter vehicle ID: ");
        String vid = input.nextLine();

        System.out.print("Enter plate number: ");
        String plate = input.nextLine();

        System.out.print("Enter type (Economy/SUV/Luxury/Electric): ");
        String type = input.nextLine();

        System.out.print("Enter capacity: ");
        int cap = getIntInput();

        if (cap == -1) {
            break;
        }
    }
```



```
if (cap <= 0) {  
    System.out.println("Invalid capacity. Must be greater than 0.");  
    break;  
}  
  
System.out.print("Enter transmission (Manual/Automatic): ");  
String trans = input.nextLine();  
  
System.out.print("Enter fuel type (Gas/Electric/Hybrid): ");  
String fuel = input.nextLine();  
  
System.out.print("Enter price: ");  
double price;  
  
try {  
    price = Double.parseDouble(input.nextLine());  
} catch (NumberFormatException e) {  
    System.out.println("Invalid input. Please enter a valid price.");  
    break;  
}  
  
if (price <= 0) {  
    System.out.println("Invalid price. Must be greater than 0.");  
    break;  
}
```




```
price = Math.round(price * 100.0) / 100.0;
```

```
ArrayList<String> dates = new ArrayList<>();
```

```
System.out.print("Enter available dates (format: YYYY-MM-DD,YYYY-MM-DD,...): ");
```

```
String datesInput = input.nextLine();
```

```
if (!datesInput.trim().isEmpty()) {
```

```
    String[] dateArray = datesInput.split(",");
```

```
    for (String d : dateArray) {
```

```
        String date = d.trim();
```

```
        if (!isValidDateFormat(date)) {
```

```
            System.out.println("Invalid date format. Use YYYY-MM-DD.");
```

```
            break;
```

```
        }
```

```
        dates.add(date);
```

```
    }
```

```
}
```

```
if (dates.isEmpty()) {
```

```
    System.out.println("Vehicle must have at least one available date.");
```

```
    break;
```

```
}
```



```
Vehicle v = null;
```

```
switch (type.toLowerCase()) {
```

```
    case "economy":
```

```
        v = new Economy(vid, plate, cap, trans, fuel, price, "Available", dates);
```

```
        break;
```

```
    case "suv":
```

```
        v = new SUV(vid, plate, cap, trans, fuel, price, "Available", dates);
```

```
        break;
```

```
    case "luxury":
```

```
        v = new Luxury(vid, plate, cap, trans, fuel, price, "Available", dates);
```

```
        break;
```

```
    case "electric":
```

```
        v = new Electric(vid, plate, cap, trans, fuel, price, "Available", dates);
```

```
        break;
```

```
    default:
```

```
        System.out.println("Invalid vehicle type.");
```

```
        break;
```

```
}
```

```
if (v == null) {
```

```
    break;
```

```
}
```

```
system.addVehicle(v);
```



```
break;
```

case 2:

```
System.out.print("Enter vehicle ID to remove: ");
```

```
String removeId = input.nextLine();
```

```
system.removeVehicle(removeId);
```

```
break;
```

case 3:

```
System.out.print("Enter Vehicle ID to update: ");
```

```
String updateVehicleId = input.nextLine();
```

```
Vehicle vehicleToUpdate = null;
```

```
for (Vehicle vehicle : system.getVehicles()) {
```

```
    if (isEqual(vehicle.getVehicleID(), updateVehicleId)) {
```

```
        vehicleToUpdate = vehicle;
```

```
        break;
```

```
    }
```

```
}
```

```
if (vehicleToUpdate == null) {
```

```
    System.out.println("Vehicle not found.");
```

```
    break;
```

```
}
```



```
System.out.print("Enter new capacity: ");

int newCapacity = getIntInput();

if (newCapacity == -1) {
    break;
}

if (newCapacity <= 0) {
    System.out.println("Invalid capacity. Must be greater than 0.");
    break;
}

System.out.print("Enter new transmission (Manual / Automatic): ");
String newTransmission = input.nextLine();

System.out.print("Enter new fuel type (Gas / Electric / Hybrid): ");
String newFuelType = input.nextLine();

System.out.print("Enter new daily rate: ");
double newDailyRate;

try {
    newDailyRate = Double.parseDouble(input.nextLine());
} catch (NumberFormatException e) {
    System.out.println("Invalid input. Please enter a valid price.");
}
```



```
        break;
    }
    if (newDailyRate <= 0) {
        System.out.println("Invalid price. Must be greater than 0.");
        break;
    }
    newDailyRate = Math.round(newDailyRate * 100.0) / 100.0;

    system.updateVehicleSpecs(vehicleToUpdate, newCapacity, newTransmission,
newFuelType, newDailyRate);
    break;

case 4:
    running = false;
    break;
default:
    System.out.println("Invalid choice.");
}
}
}

// Reads integer input safely and prevents input errors
private int getIntInput() {
    try {
        return Integer.parseInt(input.nextLine().trim());
    } catch (NumberFormatException e) {
        System.out.println("Please enter a number.");
        return -1;
    }
}
```



```
    }  
}  
  
// Reads data input safely and prevents input errors  
private boolean isValidDateFormat(String date) {  
    if (date == null || date.length() != 10) {  
        return false;  
    }  
    if (date.charAt(4) != '-' || date.charAt(7) != '-') {  
        return false;  
    }  
    for (int i = 0; i < date.length(); i++) {  
        if (i == 4 || i == 7) {  
            continue;  
        }  
        if (!Character.isDigit(date.charAt(i))) {  
            return false;  
        }  
    }  
    return true;  
}  
  
// Compares two strings while ignoring spaces and letter case  
private boolean isEqual(String a, String b) {  
    return a.replace(" ", "").equalsIgnoreCase(b.replace(" ", ""));  
}  
}
```



File 1 (vehicle.txt):

EC1|RMA1515|Economy|5|Automatic|Gas|100|Available|2026-12-20,2026-12-21,2026-12-22,2026-12-23,2026-12-24,2026-12-25,2026-12-26,2026-12-27

EC2|ABC111|Economy|5|Manual|Gas|90|Available|2026-12-22,2026-12-23,2026-12-24,2026-12-25,2026-12-26

EC3|KSA303|Economy|4|Automatic|Gas|95|Available|2026-12-20,2026-12-21,2026-12-24,2026-12-25,2026-12-28,2026-12-29

SUV1|RYD2020|SUV|7|Automatic|Gas|180|Available|2026-12-23,2026-12-24,2026-12-25,2026-12-26,2026-12-27

SUV2|JED3030|SUV|7|Automatic|Hybrid|200|Rented|

SUV3|DMM7070|SUV|6|Automatic|Gas|175|Available|2026-12-20,2026-12-21,2026-12-22,2026-12-26,2026-12-27,2026-12-28

LX1|VIP999|Luxury|5|Automatic|Gas|300|Available|2026-12-22,2026-12-23,2026-12-24,2026-12-25,2026-12-26

LX2|VIP888|Luxury|5|Automatic|Hybrid|320|Under Maintenance|

LX3|LUX555|Luxury|4|Automatic|Gas|350|Available|2026-12-20,2026-12-23,2026-12-24,2026-12-27,2026-12-28,2026-12-29

LX4|LUX477|Luxury|4|Automatic|Gas|350|Available|2026-12-20,2026-12-23,2026-12-24,2026-12-27,2026-12-28

EL1|ELE123|Electric|5|Automatic|Electric|150|Available|2026-12-22,2026-12-23,2026-12-24,2026-12-25

EL2|ELE456|Electric|5|Automatic|Electric|160|Available|2026-12-20,2026-12-21,2026-12-24,2026-12-25,2026-12-26,2026-12-27

EL3|EV900|Electric|4|Automatic|Electric|140|Available|2026-12-21,2026-12-22,2026-12-23,2026-12-28,2026-12-29



File 2 (users.txt):

CUSTOMER|Khaled|C1|0551111111|khaled@gmail.com|1995-01-01|Riyadh|SA-111111|Private|2030-01-01

CUSTOMER|Abdulrahman|C2|0552222222|abdul@gmail.com|1993-05-10|Jeddah|SA-222222|Private|2031-05-10

CUSTOMER|Suhail|C3|0553333333|suhail@gmail.com|1998-08-20|Dammam|SA-333333|Private|2032-08-20

CUSTOMER|Nasser|C4|0554444444|nasser@gmail.com|1990-03-15|Riyadh|SA-444444|Private|2030-03-15

CUSTOMER|Fahad|C5|0555555555|fahad@gmail.com|1992-07-07|Jeddah|SA-555555|Private|2031-07-07

CUSTOMER|Sara|C6|0556666666|sara@gmail.com|1997-12-12|Khobar|SA-666666|Private|2032-12-12

CUSTOMER|Omar|C7|0557777777|omar@gmail.com|1994-04-04|Makkah|SA-777777|Private|2030-04-04

STAFF|S1|Ahmed

STAFF|S2|Omar

STAFF|S3|Yousef

STAFF|S4|Mona

STAFF|S5|Sara



File 3 (bookings.txt):

B1|C1|SUV2|2026-12-20|2026-12-22|Upcoming|none

B2|C2|EC1|2026-12-27|2026-12-28|Cancelled|2026-12-24

B3|C3|LX2|2026-12-18|2026-12-19|Past|none

B4|C4|EL1|2026-12-20|2026-12-21|Upcoming|none

B5|C5|SUV1|2026-12-20|2026-12-22|Past|none

B6|C6|EC2|2026-12-20|2026-12-21|Upcoming|none

B7|C7|LX1|2026-12-20|2026-12-21|Cancelled|2026-12-18

B8|C1|EL2|2026-12-22|2026-12-23|Past|none

B9|C2|LX3|2026-12-21|2026-12-22|Upcoming|none

B10|C3|EC3|2026-12-22|2026-12-23|Cancelled|2026-12-20

B11|C1|LX4|2026-12-23|2026-12-24|Upcoming|none



Conclusion:

In conclusion, the Vehicle Rental System successfully demonstrates the practical application of Object-Oriented Programming concepts using Java. The system provides a complete solution for managing vehicle rentals, including booking, cancellation, customer management, and business analytics.

Throughout the project, key OOP principles such as encapsulation, inheritance, polymorphism, and abstraction were effectively implemented, resulting in a well-structured and maintainable system. Additionally, the use of file handling ensured data persistence across sessions, meeting the system requirements.

Overall, this project enhanced our understanding of designing real-world applications, improved our problem-solving skills, and strengthened our ability to work collaboratively as a team.